Лабораторная работа 2

Обоснование темы выпускной квалификационной работы

Разработка системы автоматизированного преобразования дизайн-макетов в структурированную конфигурацию CMS-страницы

Цель работы – обосновать исследование автоматической сборки страниц из дизайн-макетов, проанализировать литературу и аналоги, определить метод и критерии его оценки.

Основные мотивы работы – экономия времени и ресурсов при переносе макета в CMS и соблюдение требований конфиденциальности при обработке внутреннего кода и материалов. Система реализована: с её помощью собраны 12 макетов, по одному дню на каждый. Суммарное время сборки составило 12 дней вместо расчётных 60 дней при прежнем сроке пять дней на макет. Экономия составила 48 дней, или 80 %.

1 Обзор литературы по теме исследования

1.1 Исследуемая задача и основные подходы

Рассматривается преобразование готового дизайн-макета в конфигурацию компонентной CMS[1]. На вход поступают дерево элементов, свойства компонентов, тексты, графические ресурсы и изображения макета. На выходе требуется редактируемое описание страницы, совместимое с существующей библиотекой компонентов[2].

В литературе выделяются генерация интерфейса по изображению [1, 2, 4], совместное использование изображения и метаданных [3], итеративная корректировка по визуальной обратной связи [5], а также предметно-ориентированное моделирование и генерация по формальным описаниям [8]. Для выбранной задачи предлагается объединить декларативные правила преобразования известных компонентов с контролируемой нейросетевой проверкой результата.

[1] Компонентная CMS формирует страницу по структурированному описанию блоков, их параметров и вложенного содержимого. Конфигурация передаётся в формате JSON.

[2] Библиотека компонентов содержит готовые элементы интерфейса с заданными параметрами и правилами композиции. В исследовании она рассматривается как неизменяемая зависимость.

1.2 Исследователи и разработчики

Tony Beltramelli. Автор pix2code – ранней работы о генерации описания интерфейса по его изображению. Значим для анализа возможности автоматизировать перенос визуального представления в формальный формат [1].

Chenglei Si. Соавтор Design2Code, посвящённой оценке генерации веб-интерфейсов по скриншотам и сравнению результата с эталонным изображением [2].

Zhen Yang. Соавтор UI2Code^{n}; исследует итеративную генерацию и исправление интерфейса с использованием результата его визуализации [5].

С. М. Авдошин. Соавтор обзора нейросетевых методов анализа и генерации кода. Работа полезна для систематизации подходов к автоматизации рутинных операций разработки [6].

Л. Н. Лядова. Соавтор исследования, объединяющего предметно-ориентированное моделирование и генеративный ИИ для разработки средств визуализации. Этот подход близок к сочетанию формальных описаний и модели в разрабатываемой системе [8].

1.3 Организации

Stanford University. Исследовательская группа SALT участвует в разработке Design2Code и инструментов оценки визуально обусловленной генерации веб-страниц [2].

Национальный исследовательский университет «Высшая школа экономики». Проводит исследования нейросетевой генерации кода и предметно-ориентированного моделирования [6, 8].

Anima. Разрабатывает средства преобразования макетов в код и сопоставления элементов дизайна с компонентами существующего проекта [12, 13].

Builder.io. Разрабатывает средства генерации интерфейсов с индексированием компонентов и токенов дизайн-системы проекта [14].

Tilda Publishing. Предоставляет импорт элементов и макетов Figma в визуальный редактор Zero Block [10].

1.4 Конференции

ACM EICS. Симпозиум по инженерии интерактивных компьютерных систем; в 2018 году опубликована работа pix2code [1].

NAACL. Конференция по компьютерной лингвистике и языковым технологиям; в 2025 году опубликована работа Design2Code [2].

ГрафиКон. Международная конференция по компьютерной графике и машинному зрению; в материалах 2025 года рассматривается интеграция предметно-ориентированного моделирования и генеративного ИИ [8].

1.5 Научные журналы

«Информационные технологии». Публикует исследования по автоматизации разработки; пример – обзор нейросетевых методов анализа и генерации кода [6].

«Вестник молодых учёных СПбГУПТД». Содержит публикации по цифровым технологиям и веб-дизайну; пример – применение ИИ при подготовке макетов веб-сайтов [7].

«Труды Института системного программирования РАН». Публикует работы по программной инженерии и тестированию; для темы значимы методы сокращения трудоёмкости проверки графического интерфейса [9].

1.6 Источники и их значение для исследования

Основу обзора составляют пять англоязычных научных работ [1–5] и пять русскоязычных источников [6–10]. Среди русскоязычных источников четыре научные публикации и одна официальная техническая документация. Дополнительные источники [11–15] описывают интеграции, аналоги и классификацию темы. Дата обращения к электронным ресурсам – 30.10.2026.

[1] Beltramelli T. pix2code: Generating Code from a Graphical User Interface Screenshot // Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems. 2018. P. 1–6. DOI: 10.1145/3220134.3220135. URL: https://doi.org/10.1145/3220134.3220135

Предлагается генерация формального описания интерфейса по изображению. Работа задаёт историческую основу направления; её постановка не охватывает объединение адаптивных макетов в конфигурацию существующей CMS.

[2] Si C., Zhang Y., Li R., Yang Z., Liu R., Yang D. Design2Code: Benchmarking Multimodal Code Generation for Automated Front-End Engineering // Proceedings of NAACL 2025. Vol. 1. P. 3956–3974. DOI: 10.18653/v1/2025.naacl-long.199. URL: https://aclanthology.org/2025.naacl-long.199/

Описаны набор из 484 веб-страниц и оценка восстановления визуальных элементов и их расположения. Полезен для выбора показателей качества, но эталонный HTML не является конфигурацией целевой CMS.

[3] Gui Y., Zhang J., Wang Y. et al. Figma2Code: Automating Multimodal Design to Code in the Wild. 2026. arXiv:2604.13648. DOI: 10.48550/arXiv.2604.13648. URL: https://arxiv.org/abs/2604.13648

Исследуется использование изображений вместе с метаданными Figma. Отмечаются ограничения адаптивности и сопровождаемости результата. Это обосновывает необходимость оценивать структуру конфигурации наряду с внешним видом.

[4] Laurençon H., Tronchon L., Sanh V. Unlocking the conversion of Web Screenshots into HTML Code with the WebSight Dataset. 2024. arXiv:2403.09029. DOI: 10.48550/arXiv.2403.09029. URL: https://arxiv.org/abs/2403.09029

Представлен синтетический набор из 2 млн пар HTML и скриншотов. Полезен как пример подготовки данных; для исследования CMS требуется дополнительно размечать компоненты, адаптивные варианты и целевые конфигурации.

[5] Yang Z., Hong W., Xu M. et al. UI2Code^{n}: UI-to-Code Generation as Interactive Visual Optimization. 2026. arXiv:2511.08195v4. DOI: 10.48550/arXiv.2511.08195. URL: https://arxiv.org/abs/2511.08195v4

Генерация рассматривается как повторяющийся цикл выполнения, визуальной проверки и исправления. Работа обосновывает использование обратной связи; в данной ВКР изменения ограничиваются конфигурацией существующих компонентов.

[6] Авдошин С. М., Арутюнов Г. А. Обзор нейросетевых методов анализа и генерации кода // Информационные технологии. 2022. Т. 28. № 7. С. 378–391. DOI: 10.17587/it.28.378-391. URL: https://publications.hse.ru/articles/651384234

Систематизируются нейросетевые подходы к анализу и генерации программного кода. Источник используется для обоснования автоматизации повторяющихся операций и выбора места модели в процессе разработки.

[7] Верстунина Е. А. Применение искусственного интеллекта в современной верстке веб-сайтов // Вестник молодых ученых Санкт-Петербургского государственного университета промышленных технологий и дизайна. 2024. № 1. С. 125–128. URL: https://publish.sutd.ru/docs/content/vestnik_mu_1_2024.pdf#page=125

Рассматриваются инструменты ИИ для подготовки веб-макетов на примере Figma и Uizard. Полезен для описания работы дизайнера; вопросы типизированной CMS-конфигурации и её проверки остаются за рамками этой публикации.

[8] Джейранян А. Д., Ларионова Я. А., Лядова Л. Н. Проектирование инструментов визуализации данных на основе интеграции возможностей предметно-ориентированного моделирования и генеративного искусственного интеллекта // ГрафиКон 2025. С. 353–366. DOI: 10.25686/978-5-8158-2474-4-2025-353-366. URL: https://www.graphicon.ru/html/2025/papers/paper_033.pdf

Предложено сочетание предметно-ориентированных моделей и генеративных средств. Работа относится к смежной задаче визуализации данных и поддерживает выбор гибридной архитектуры, но не доказывает её эффективность для сборки CMS-страниц.

[9] Баранцев А. В., Грошев С. В., Омельченко В. А. Генерация оптимизированных для ручного выполнения сценариев тестирования приложений с графическим интерфейсом пользователя // Труды ИСП РАН. 2009. Т. 17. С. 75–92. URL: https://www.ispras.ru/proceedings/isp_17_2009/isp_17_2009_75/

Рассматривается построение сценариев проверки интерфейса по модели состояний. Источник полезен для проверки вкладок, слайдеров и переходов: сходство одного скриншота не подтверждает корректность всех состояний.

[10] Tilda Publishing. Импорт из Figma // Справочный центр Tilda. Электронный ресурс. URL: https://help-ru.tilda.cc/zero/figma-import (дата обращения: 30.10.2026).

Описан перенос макетов и слоёв в Zero Block. Это практический аналог сокращения ручного переноса; его результат связан с моделью редактора Tilda, а не с заданной схемой сторонней компонентной CMS.

[11] Figma. File endpoints // Developer Docs. Электронный ресурс. URL: https://developers.figma.com/docs/rest-api/file-endpoints/ (дата обращения: 30.10.2026).

Документация описывает доступ к структуре файлов и отдельным узлам. Используется для определения входных данных и воспроизводимого получения метаданных макета.

[12] Anima. Code settings // Anima Documentation. Электронный ресурс. URL: https://docs.animaapp.com/docs/1-code-settings (дата обращения: 30.10.2026).

Документирует выходные технологии, настройки стилей и адаптивных точек. Используется при сравнении целевого формата результата.

[13] Anima. Component Matching // Anima Documentation. Электронный ресурс. URL: https://docs.animaapp.com/docs/component-matching-copy (дата обращения: 30.10.2026).

Описывает поиск соответствий между макетом и компонентами существующего проекта. Подтверждает, что повторное использование компонентов реализуется и в аналогах.

[14] Builder.io. Design system indexing // Builder Documentation. Электронный ресурс. URL: https://www.builder.io/c/docs/component-indexing (дата обращения: 30.10.2026).

Описывает индексирование компонентов, иконок и токенов проекта для генерации интерфейса. Используется для сравнения подходов к интеграции дизайн-системы.

[15] UDC Consortium. 004.4 Программные средства // UDC Summary. Электронный ресурс. URL: https://udcsummary.info/php/index.php?id=13871&lang=ru (дата обращения: 30.10.2026).

Официальный сокращённый классификатор; используется для выбора рубрики 004.4 и подраздела инструментальных средств разработки программного обеспечения.

1.7 Выводы по обзору литературы

Преобразование макета и доработка по скриншотам уже исследуются. Вклад ВКР связан с получением компактной конфигурации заданной CMS из нескольких адаптивных макетов при неизменяемых компонентах. Оценивать необходимо сохранность содержимого, корректность схемы, дублирование и время ручной доводки.

Выбраны декларативные адаптеры, поблочное сопоставление адаптивов, структурная и типовая проверка, визуальная обратная связь. Гибридный подход сравнивается с ручной сборкой и отдельным использованием правил либо модели. Эксперимент должен установить вклад каждого этапа в экономию ресурсов.

2 Обзор и анализ аналогичных решений

Прямые функциональные аналоги – Anima и Builder.io: они преобразуют дизайн в код и используют готовые компоненты проекта. Косвенный аналог – Tilda с импортом Figma в Zero Block. Она сокращает ручной перенос, используя собственную модель визуального редактора. Эти решения отличаются целевым форматом и способом интеграции [10, 12–14].

Сравнение основано на документированных сценариях на 30.10.2026. Оценка необходимой интеграции с целевой CMS является аналитической; сравнительные испытания продуктов не проводились.

Таблица 1 – Сравнение аналогов

Критерий	**Anima**	**Builder.io**	**Tilda**	**Разрабатываемая система**
Основной результат	Код React, Vue или HTML	Код с учётом дизайн-системы	Элементы Zero Block	JSON по схеме целевой CMS

Критерий	Anima	Builder.io	Tilda	Разрабатываемая система
Готовые компоненты	Component Matching	Индекс компонентов	Элементы редактора	Разрешённый каталог CMS-компонент из специфичной дизайн-системы
Объединение адаптивов в целевой CMS	Нужна отдельная интеграция	Нужна отдельная интеграция	Другая модель страницы	Общие блоки и изменяемые параметры
Дальнейшая работа	Проверка и интеграция кода	Проверка в проекте	Правки в Zero Block	Проверка и правка конфигурации
Зависимость от целевой схемы	Схема CMS не является форматом вывода	Нужна настройка вывода под CMS	Формат платформы Tilda	Схема входит в контракт сборки

Источники характеристик: Anima [12, 13], Builder.io [14], Tilda [10]. Для разрабатываемой системы указаны архитектурные решения, а не результаты сравнительного тестирования.

2.1 Выводы по сравнению аналогов

Аналоги уже автоматизируют значительную часть переноса дизайна и поддерживают повторное использование компонентов. Основанием для собственной системы является потребность встроить автоматизацию в существующий процесс работы с CMS: выдавать её штатный формат, сохранять редактируемость содержимого и уменьшать дублирование между разрешениями. Отдельного исследования требует баланс между экономией ручного труда, стоимостью модельных вызовов и качеством результата.

Дополнительное условие выбора – допустимость обработки закрытых материалов. В рассматриваемом проекте используется собственная CMS с учётом требований конфиденциальности и внутренних интеграций. Поэтому автосборщик должен работать с её форматом и разрешёнными средствами обработки. Применимость внешнего продукта требует отдельной оценки размещения, доступа к данным и совместимости; наличие нужных функций само по себе её не подтверждает.

3 Итоги и постановка исследования

Тема ВКР. Разработка системы автоматизированного преобразования дизайн-макетов в структурированный пейлоад CMS-страницы.

УДК для тезиса. 004.4’2 – инструментальные средства разработки программного обеспечения [15].

Ключевые слова. дизайн-макет, CMS, автоматизированная сборка, генерация конфигураций, декларативные адаптеры, мультимодальные модели, адаптивный интерфейс, визуальная проверка, трудозатраты, конфиденциальность.

3.1 Актуальность исследования

Основная актуальность исследования связана с экономией времени и ресурсов при подготовке веб-страниц. В традиционном процессе дизайнер создаёт макет, после чего контент-менеджер вручную воспроизводит его в CMS: выбирает блоки, переносит тексты и изображения, настраивает параметры и адаптивные варианты. Таким образом, уже зафиксированные в макете структура и содержимое повторно вводятся в другую систему. Эта работа занимает время специалиста и увеличивает срок готовности лендинга.

В реализованном процессе ручная сборка перестаёт быть обязательным этапом. Дизайнер подготавливает макет, система преобразует его в конфигурацию страницы, а контент-менеджер проверяет результат и при необходимости разбирает исключения. Для поддерживаемых макетов его участие может быть ограничено итоговой проверкой. Это позволяет высвободить время на содержательные задачи и уменьшить зависимость сроков запуска от доступности исполнителя ручной сборки.

Практическую значимость подтверждает применение реализованной системы для сборки 12 макетов. Каждый макет был собран за один день; суммарное время составило 12 дней. При прежнем сроке пять дней на макет аналогичный объём потребовал бы 60 дней. Таким образом, расчётная экономия относительно прежнего процесса составляет 48 дней, или 80 %, а суммарное время сборки уменьшилось в пять раз. Данные описывают уже выполненную работу: 12 дней относятся к фактической сборке, 60 дней – к оценке того же объёма по прежнему процессу.

Второй основной мотив – конфиденциальность исходных материалов. В рассматриваемом проекте используется внутренний код организации, на который распространяются обязательства о неразглашении (NDA). Макеты, описания компонентов и конфигурации также могут содержать закрытую информацию. Их передача на сторонние площадки без предусмотренного организацией согласования недопустима по условиям проекта. Эти ограничения стали одной из причин использования собственной CMS. Автосборщик должен поддерживать её формат и обрабатывать данные в разрешённой среде, включая обращения к нейросетевой модели. Собственная реализация позволяет управлять этим процессом, но соблюдение требований зависит от настроек доступа и используемых интеграций.

Экономия возникает за счёт автоматического переноса содержимого, выбора известных компонентов и согласования адаптивных вариантов. Дополнительный эффект связан с уменьшением повторных исправлений: пропущенные изображения, незаполненные вкладки, неверные параметры и потеря текста выявляются до передачи результата на итоговую проверку. При наличии общего содержимого его хранение в одном месте также уменьшает трудоёмкость последующих изменений страницы.

Современные мультимодальные модели создают предпосылки для такой автоматизации, однако исследования показывают сохраняющиеся ошибки восстановления элементов, компоновки и адаптивности [2, 3]. Полностью генеративный подход требует контроля допустимых компонентов и параметров. Детерминированные правила дают более предсказуемый результат для известных структур, но не охватывают все варианты дизайнерских макетов. Поэтому актуально исследование их сочетания с нейросетевой проверкой и визуальной обратной связью [5, 6].

Практическая цель состоит в сокращении обязательного ручного участия при сохранении корректности и сопровождаемости страницы. Для оценки этого эффекта необходимо учитывать время проверки и исправлений, стоимость вычислений и сопровождения адаптеров. Результатом исследования должна стать обоснованная методика преобразования макетов и измерение условий, при которых

автоматическая сборка действительно экономит ресурсы по сравнению с ручным процессом.

3.2 Цель задачи и исследовательская гипотеза

Цель исследования. Разработать и экспериментально оценить систему преобразования дизайн-макетов в пейлоад CMS-страниц, сокращающую время подготовки и объём ручной работы контент-менеджера при соблюдении требований конфиденциальности, контрактов компонентов и контроле визуального соответствия.

Объект исследования. Процесс переноса готовых дизайн-макетов в компонентную систему управления контентом.

Предмет исследования. Методы декларативного преобразования компонентов, объединения адаптивных вариантов и корректировки конфигурации на основе структурной и визуальной обратной связи.

Гипотеза. Совместное применение адаптеров и контролируемой нейросетевой корректировки уменьшает время ручной доводки по сравнению с использованием каждого подхода отдельно при сопоставимом качестве страниц.

1. Проанализировать ручной процесс сборки, литературу, аналоги и основные источники затрат.

2. Определить структуру входных данных, целевую схему CMS, разрешённую среду обработки и критерии качества результата.

3. Разработать декларативные адаптеры компонентов и поблочное объединение адаптивных вариантов.

4. Реализовать структурную и типовую валидацию, сохранение происхождения содержимого и контролируемую нейросетевую корректировку.

5. Организовать визуальную проверку блоков на заданных разрешениях и в интерактивных состояниях без изменения исходного кода компонентов.

6. Провести сравнительный эксперимент, измерить длительность, трудозатраты, стоимость вычислений и качество; определить ограничения и условия применимости.

3.3 Полный процесс преобразования макета

Границы ответственности фиксированы: адаптер извлекает данные своего компонента; общий сборщик объединяет блоки и варианты; модель предлагает исправления конфигурации; валидатор проверяет допустимость; человек принимает результат. Библиотека компонентов и её тесты не изменяются под отдельный макет. Визуальный цикл рассматривается как проверяемая часть метода, а не как гарантия полного совпадения любой страницы.

1 Выбор исходных макетов. Определяются допустимые данные, среда обработки и способ обращения к модели. Указываются корневой макет страницы и связанные разрешения. Языковые, региональные версии и дополнительные состояния вкладок или слайдеров определяются отдельно. Язык текста сам по себе не задаёт регион публикации.

2 Получение исходных данных. Через API загружаются дерево элементов, свойства компонентов, тексты, стили и ссылки на ресурсы [11]. Сохраняются снимок исходных данных и эталонные изображения, необходимые для повторения проверки.

3 Выделение блоков. Структура страницы преобразуется в последовательность смысловых блоков. Для каждого блока сохраняются связи с исходными узлами, чтобы проверять происхождение текста, изображений и предложенных изменений.

4 Первичная сборка адаптерами. Для известного компонента используется описание в виде дерева правил: признаки распознавания, источники полей, соответствия параметров и вложенные элементы. Один общий интерпретатор применяет эти описания; специальные случаи остаются в адаптере владельца компонента.

5 Объединение адаптивных вариантов. Блоки сопоставляются между разрешениями. Общие тексты и параметры сохраняются один раз; различия записываются в поддерживаемые адаптивные поля. Отдельные ветви создаются только при различии композиции, которое нельзя представить параметрами.

6 Подготовка конфигурации. Формируются ресурсы, локализуемые значения и подтверждённые региональные варианты. Система проверяет типы, вложенность, заполненность составных блоков и сохранность содержимого. Отсутствующие данные попадают в диагностику.

7 Визуальная проверка. Конфигурация отображается штатными компонентами. Для каждого заданного разрешения и состояния сопоставляются эталонный блок и скриншот результата. Через разрешённый способ доступа модель получает необходимые для блока изображения, участок JSON и допустимые параметры компонентов. Разрешение на обработку проверяется и для скриншотов, и для текстового контекста.

8 Корректировка и повторная проверка. Модель предлагает локальное изменение конфигурации. Проверяются схема, происхождение содержимого и разрешённые границы изменения. После принятия выполняются повторная визуализация и проверка затронутых адаптивов и состояний.

9 Завершение и итоговый контроль. Итерации заканчиваются при выполнении критериев качества, отсутствии улучшений либо исчерпании бюджета. Повторяющиеся значения сокращаются без изменения смысла. Ограничения и нерешённые расхождения передаются на проверку человеку.

10 Подготовка к публикации. После приёмки изображения и локализуемые значения размещаются в инфраструктуре контента[3], ссылки обновляются и конфигурация передаётся в CMS. Публикация отделена от генерации и экспериментальных исправлений.

[3] Инфраструктура контента включает хранилище статических ресурсов и сервис локализации, который хранит текстовые значения и их языковые варианты.

3.4 Данные и сервисы для исследования

Собственный набор. Обезличенные макеты лендингов, их адаптивные варианты, изображения, состояния интерактивных блоков и эталонные CMS-конфигурации. Для каждой страницы фиксируются версия библиотеки компонентов, дата снимка, параметры запуска и результаты ручной проверки. Для публичных экспериментов подготавливаются нейтральные тексты, разрешённые к распространению изображения и демонстрационные контракты. Закрытый код и внутренние идентификаторы не публикуются; замена названий сама по себе не подтверждает допустимость публикации.

Открытые данные. Design2Code содержит 484 веб-страницы для визуальной оценки [2]; WebSight – 2 млн синтетических пар HTML и скриншотов [4]; Figma2Code – макеты с метаданными, включая отобранный набор из 213 случаев [3]. Эти наборы помогают исследовать визуальное восстановление, но не содержат готовых конфигураций разработанной нами CMS.

Экспериментальная среда. Разрешённые организацией API дизайн-редактора и способ доступа к мультимодальной модели, локальный сборщик, средство проверки схемы и браузер для визуализации. Версии инструментов и модели фиксируются. Все адаптивы и состояния одной страницы включаются в одну группу при разделении данных, чтобы исключить проверку на варианте уже использованного макета.

3.5 Оценка экономии времени ресурсов и качества

Система реализована и применена для сборки 12 макетов, каждый из которых собран за один день. Результат сопоставляется с прежним сроком пять дней на макет. В таблице 2 сравнивается одинаковый объём работ – 12 макетов.

Таблица 2 – Зафиксированный результат практического применения

Показатель	По прежнему процессу	С автосборкой
Количество макетов	12	12 собраны
Время на один макет	5 дней	1 день
Суммарное время сборки	60 дней (12 × 5)	12 дней (12 × 1)
Экономия времени	База сравнения	48 дней (60 − 12)

Показатель	По прежнему процессу	С автосборкой
Сокращение времени	База сравнения	80 %; в 5 раз

Источник таблицы 2 – данные автора о 12 выполненных сборках. 60 дней – расчёт для прежнего процесса, 12 дней – сумма времени фактических сборок. Суммы сроков отдельных сборок не обозначают календарную длительность всей серии при параллельной работе.

Дальнейшая экспериментальная оценка направлена на определение вклада отдельных этапов уже реализованной системы. Для этого сопоставляются четыре режима: ручная сборка; только адаптеры; только модель; адаптеры с нейросетевой корректировкой. Используются одинаковые макеты и ограничения компонентов. Фиксируются время подготовки, активное время человека, время модельных вызовов, стоимость вычислений и число итераций. Учитываются также проверка, исправления и повторная приёмка.

Качество оценивается отдельно по доле валидных конфигураций, сохранности текстов и ресурсов, работе интерактивных состояний, визуальному соответствию и дублированию данных. Визуальная проверка выполняется при одинаковых размерах окна, шрифтах и состоянии интерфейса. Применяются заранее выбранные показатели геометрии и изображения, дополненные экспертной оценкой [2]. Модель проходит по каждому блоку, делая сравнение и выравнивая.

Ресурсная экономия определяется разностью стоимости ручного и автоматизированного процессов: учитываются фактические человеко-часы по ролям, стоимость вычислений и доля затрат на сопровождение инструмента. Для серийного применения отдельно измеряется влияние сложности макета и числа адаптивов. Итогом должны стать границы применимости системы и оценка того, в каких случаях участие контент-менеджера действительно сводится к проверке результата.